ECOLE POLYTECHNIQUE UNIVERSITAIRE DE NICE SOPHIA

UNIVERSITÉ CÔTE D'AZUR

POLYTECH

Département Robotique Autonome

Maxime DELUBAC-LOPPI--DUMORTIER

Robot-bras PX-100 et Caméra D435

Sommaire

I/ Introduction **3**
II/ Robot-bras **4**
1) Cahier des charges 4
2) Composants 4
3) Détection 4
4) Microcontrôleur 5
5) Taille et distance 6
6) Suite et améliorations 6
III/ Caméra de profondeur D435 ..... **7**
1) Cahier des charges 7
2) Caméra D435 7
3) Choix du modèle et de l'environnement 7
4) Machine Vision 7
5) Taille et distance 8
6) Améliorations apportées 9
7) Mise en place de l'environnement ROS2 10
8) Implémentation dans ROS2 10
9) Améliorations possibles 12
IV/ Conclusion **13**
V/ Bibliographie **14**

Remerciements

Je souhaiterais remercier tous les enseignants pour cette année d'études et pour leur investissement et leur temps.

Je remercie également le personnel du Fablab et plus particulièrement Frédéric Juan, qui m'a permis de changer de projet tout en me fournissant le matériel nécessaire.

Enfin, je remercie Elena Thouny, les étudiants de ma classe qui m'ont soutenu pour ce second projet, ainsi qu'un élève récemment diplômé, Matthias Vermot-Desroches, qui m'a guidé sur l'approche à adopter.

I/ Introduction

Ce rapport présente mes projets de robotique R&D de deuxième année d'ingénieur en Robotique (master 1) à Polytech Nice Sophia.

Pour mon premier projet, j'ai réalisé un robot-bras capable de ramasser et de mettre en hauteur un objet sur le sol. Il a pour but d'aider les personnes à mobilité réduite afin qu'elles n'aient pas à se baisser. Ce projet a été effectué avec deux camarades qui ne participent pas à l'écriture de ce rapport. Dans une première partie, je développerai les aspects sur lesquels j'ai travaillé.

Pour mon second projet, je me suis servi d'une caméra de profondeur Intel RealSense D435. J'ai fait un algorithme de reconnaissance d'objets que j'ai pu implémenter dans un environnement ROS2. J'ai effectué ce projet seul et le détaillerai dans une seconde partie.

II/ Robot-bras

1) Cahier des charges

Notre objectif était de concevoir un robot capable de ramasser des objets sur le sol de manière autonome. Il devait donc être en mesure de détecter un objet, se déplacer jusqu'à lui, le ramasser et le mettre en hauteur pour l'utilisateur. Ce robot a pour but d'aider les personnes à mobilité réduite afin qu'elles puissent lui demander de ramasser n'importe quel objet à leur place.

Nous avons pensé son fonctionnement de la manière suivante : l'utilisateur, doté d'un appareil électronique (par exemple, un téléphone), télécommande le robot grâce à un retour caméra. Celui-ci sert aussi d'interface pour déplacer le robot avec des boutons. Une fois que l'objet souhaité est dans le champ de vision de la caméra, l'utilisateur clique sur l'écran pour le sélectionner et le robot va le ramasser de manière autonome.

2) Composants

Le composant le plus important est le bras robotisé. C'est un bras PX-100 dont nous avons appris le fonctionnement en ROS2, au cours de l'année. Il est équipé de cinq servomoteurs : un pour la base, trois pour le bras et un pour la pince.

Pour pouvoir effectuer ses tâches, le robot doit pouvoir "voir". Une simple caméra suffit pour faire de la reconnaissance d'objets, nous avons donc réutilisé celle qui nous avait été fournie l'année précédente, en cours de Computer Vision. Son taux de rafraîchissement est de 60 fps et sa résolution est de 720p.

Afin de ramasser un objet, nous voulions que le robot s'aligne parfaitement à celui-ci, et qu'il s'approche à une faible distance pour que le bras puisse entrer en fonction. Ainsi, nous n'avons utilisé qu'une seule caméra.

Pour trouver ses caractéristiques (taille et distance), nous avons choisi un capteur de distance à ultrasons que nous avons placé en bas du robot.

Enfin, pour le microcontrôleur, nous avons gardé la carte Nvidia Jetson Nano (qui nous avait également été fournie l'année précédente en cours de Computer Vision).

3) Détection

Mon travail a été de gérer la phase de détection d'objets. Pour cela, j'ai décidé d'utiliser un modèle pré-entraîné connu, YOLOv8. J'ai choisi la version la plus petite, YOLOv8 Nano, qui est la plus optimisée pour une Nvidia Jetson Nano.

Pour la première version de mon code, mon objectif était d'entourer les objets avec des bounding box de Yolo. Il s'agit de boîtes de détection qui délimitent la taille des objets filmés. Avec cette base, j'ai pu implémenter un système de tracking grâce à la bibliothèque CV2.

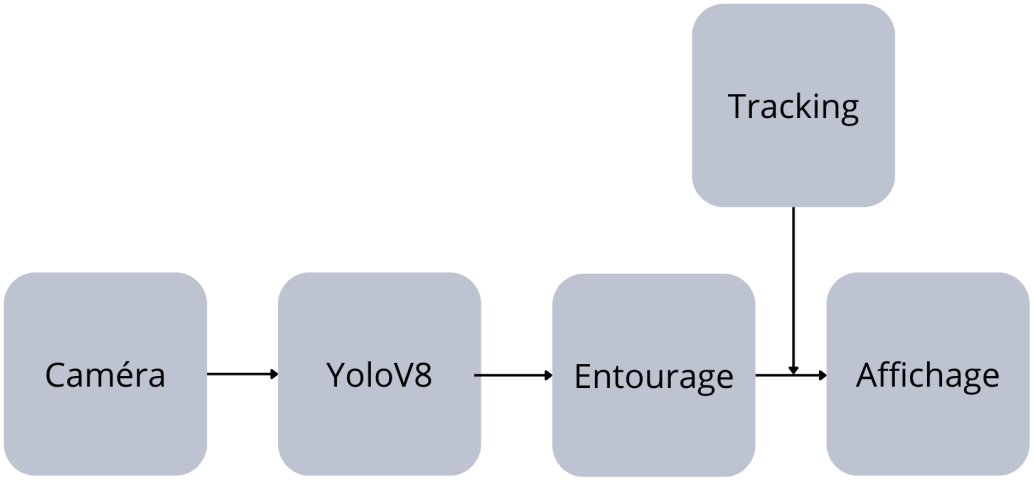

Figure 1 : Algorithme de détection

J'ai ensuite mis en relation les codes de mes camarades avec le mien (retour caméra grâce à un .html, récupération des données du capteur de distance et alignement à l'objet). Avec notre robot, nous étions donc capable de :

- Avoir un retour caméra sur un appareil (par exemple, un téléphone ou un ordinateur),
- Télécommander le robot avec cette interface,
- Détecter des objets,
- Cliquer sur l'écran de l'appareil et récupérer le pixel sélectionné (qui correspond à un objet),
- Tracker cet objet, c'est-à-dire le suivre en continu et connaître ses paramètres,
- Nous aligner au centre de la bounding box correspondante,
- Nous avancer à la distance nécessaire.

Il ne nous restait donc plus qu'à connaître la taille réelle de l'objet souhaité et à coder le mouvement du bras du robot pour le ramasser.

En ce qui concerne le retour caméra, lorsqu'on clique sur l'écran, on clique sur un pixel qui est récupéré par le code. Lors de la détection, il se peut que plusieurs objets soient superposés. Ceci a posé problème : CV2 choisit la plus grande bounding box qui contient ce pixel et non pas la plus petite, ce qui est contre-intuitif puisque nous avons tendance à appuyer sur le centre de l'objet pour le sélectionner. Ma solution a été de tracker l'objet avec le centre le plus proche du pixel cliqué (le centre de l'objet est le centre de la bounding box).

4) Microcontrôleur

La carte Nvidia Jetson Nano est une carte normalement assez puissante pour réaliser notre projet (phases de détection, tracking et retour caméra). Cependant, j'ai passé du temps à essayer de la faire fonctionner (problèmes d'horloge, etc). De plus, après avoir réussi, j'ai aussi passé du temps à essayer d'optimiser nos codes (optimisations des résolutions, optimisations du taux de rafraîchissement, changement du modèle, etc) car la

carte n'était pas en mesure de les faire tourner correctement. Même avec la plus petite version de YOLOv8 (YOLOv8 Nano) et un code basique, la carte ne fournissait pas plus de 0.2 fps sur le retour caméra, ce qui est trop peu pour du tracking d'objet. Ce problème m'a fait perdre beaucoup de temps donc j'ai décidé de garder mon ordinateur comme carte.

5) Taille et distance

Le code de détection que j'ai établi me permettant uniquement de détecter des objets en 2D, il m'a fallu me servir du capteur de distance pour obtenir la taille réelle.

On sait que le robot est aligné à l'objet s'il est aligné au centre de la bounding box. Une fois que l'alignement est fait, le capteur récupère la distance.

J'ai ensuite calculé la largeur et la hauteur. Mon objectif était de convertir ces valeurs en pixels sur l'écran à des centimètres en réalité. Pour cela, j'ai utilisé la formule :

$$X_{réel} = \frac{X*D}{F}$$

avec X la valeur en pixels, D la distance caméra-objet et F la focale de la caméra.

Il m'a ensuite fallu calibrer la caméra. Chaque caméra à sa propre focale dûe à la forme de sa lentille et ses paramètres intrinsèques. Pour trouver F, il m'a suffit d'utiliser la formule précédente dans le sens inverse pour passer de centimètres à des pixels :

- On se met à une distance connue de l'objet D,
- On mesure l'objet,
- On regarde à combien de pixels cela correspond sur l'écran,
- On isole et on calcule F dans la formule.

Lorsqu'on obtient F, le programme a aussi X et D, il peut donc retrouver la taille réelle de l'objet.

Je n'ai pas eu le temps de faire cette calibration car c'est à ce moment-là que j'ai changé de projet.

6) Suite et améliorations

Il serait intéressant d'approfondir les aspects suivants :

- Implémenter les mouvements du bras et la récupération d'un objet.
- Ajouter la possibilité de commandes vocales pour diriger le robot. Cela peut servir si l'utilisateur n'est pas en capacité d'utiliser l'interface.
- Réaliser un robot encore plus autonome qui puisse chercher seul l'objet demandé dans la pièce.
- Ajouter une plateforme qui se relèverait pour mettre l'objet en hauteur.

III/ Caméra de profondeur D435

1) Cahier des charges

Le but de ce projet était de continuer à faire de la reconnaissance d'objets mais avec une caméra bien plus performante et qui offre plus de possibilités. L'objectif principal est de pouvoir implémenter cette intelligence artificielle dans un robot-bras.

2) Caméra D435

La caméra de profondeur D435 est équipée de deux caméras infrarouges, qui lui permettent de faire une image en couleurs (chaud/froid) pour la profondeur. Elle possède aussi une caméra RGB classique. Enfin, son projecteur infrarouge lui permet d'être plus précise et plus robuste aux surfaces lisses (par exemple, un mur monochrome).

3) Choix du modèle et de l'environnement

J'ai choisi de rester sur mon environnement Python VSCode pour gagner du temps car j'y suis familiarisé. Je me suis fixé pour objectif de réussir à établir un code qui détecte correctement des objets et qui arrive à trouver de manière précise leur taille et leur distance. Pour aller plus loin et s'il me restait du temps, je voulais implémenter mes recherches dans un environnement ROS2, bien plus adapté pour la robotique.

Puisque j'ai gardé mon ordinateur personnel pour coder, j'ai choisi la meilleure version de YOLOv8 qui est YOLOv8x-segmentation. Ce modèle est plus performant que la version Nano et permet de faire des masques qui ajoutent une surface qui recouvre parfaitement l'objet, en plus des encadrements de la bounding box. J'ai installé la version Cuda de YOLOv8 qui m'a permis de faire tourner la détection sur mon GPU (RTX 3070 de Nvidia) et non sur mon CPU. Ainsi, j'ai pu garder les pleines performances de mon ordinateur pour la partie tracking.

4) Machine Vision

Dans un premier temps, pour bien comprendre le code, il faut rappeler que la caméra D435 permet d'avoir deux flux de données : un flux RGB et un flux pour la profondeur. Ces flux sont sous forme de matrices :

- Une matrice M_1, de taille $largeur * hauteur$ pour la profondeur. Chaque $M_1(i,j)$ contient la valeur de la distance de l'objet à la caméra pour ce pixel.
- Une matrice M_2, de taille $largeur * hauteur * 3$ pour le flux RGB, avec $M_2(i,j)$ qui contient les valeurs correspondantes de R, G et B pour chaque pixel.

Ces deux flux n'ont cependant pas la même résolution, j'ai donc choisi une résolution commune de 848x480.

J'ai décidé de séparer la phase de détection en deux threads différents, c'est-à-dire deux 'endroits' indépendants. L'analyse d'une frame par Yolo prend plusieurs centaines de

millisecondes, ainsi, l'affichage RGB de la caméra (60 fps) est plus rapide que l'analyse de chaque image. Il n'est possible d'afficher le retour caméra avec le masque que lorsque Yolo a fini de traiter une image, mais le rendu n'est pas suffisamment fluide.

C'est pourquoi j'ai préféré séparer l'affichage RGB et le thread de détection de Yolo. L'affichage RGB se fait en continu (60 fps) sur le thread principal. Yolo travaille, analyse de son côté et renvoie le masque sur le flux principal lorsqu'il a terminé le traitement de l'image. Cependant, si j'analysais chaque image avec Yolo, je cumulerais du retard puisque le flux RBG envoie 60 fps et que Yolo ne peut pas traiter autant d'images dans cette période de temps.

Ma solution a été de créer une queue de taille 1 :

- Une image rentre dans la queue, elle est maintenant pleine.
- L'image est analysée par Yolo.
- Les images qui arrivent pendant ce temps ne sont pas analysées car la queue est pleine. Elles sont 'oubliées', on les passe sans s'en préoccuper.
- Lorsque l'image a fini d'être traitée, on l'affiche avec le flux RGB sur le thread principal. On voit donc l'objet entouré avec la bounding box et le masque correspondants.
- La queue se vide et à ce moment-là, la frame suivante du flux RGB est à son tour analysée.

Ce système m'a donc permis d'avoir un affichage très fluide sans accumuler de retard sur les images à traiter.

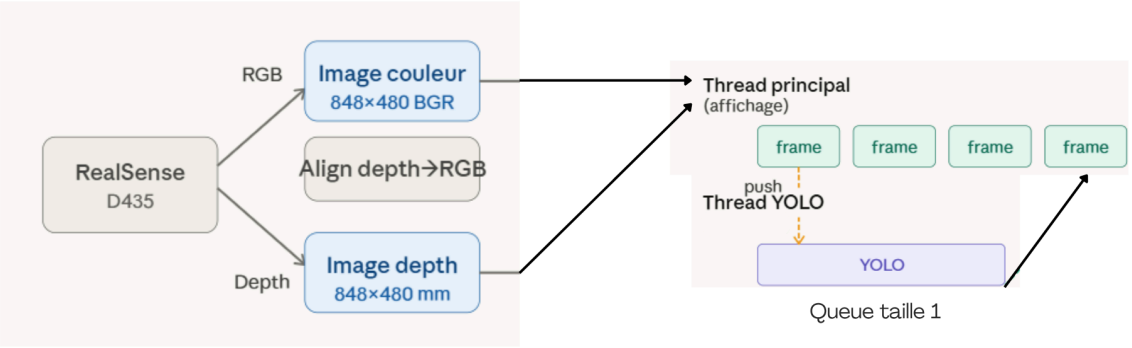

Figure 2 : Algorithme de détection

5) Taille et distance

Pour déterminer la valeur des distances, la caméra utilise ses deux rayons infrarouges pour faire une matrice (comme expliqué précédemment). Elle utilise un système de SL (Structured Light) pour trouver la profondeur : des motifs précis sont projetés sur les surfaces et chaque caméra infrarouge les détecte. Elle mesure la disparité entre les deux flux IR et grâce à une formule liant la disparité des pixels, leurs paramètres intrinsèques et la profondeur, on obtient une matrice où chaque pixel est associé à une valeur de profondeur.

Pour trouver la distance de l'objet, on récupère toutes les valeurs de profondeur des pixels qui sont dans le masque Yolo, et on calcule leur médiane. YOLOv8x-seg est utile car on prend uniquement les pixels de l'objet et non ceux qui sont dans la bounding box sans appartenir à l'objet. On calcule la médiane des valeurs de profondeur et non la moyenne car celle-ci est trop sensible aux pixels aberrants (dûs aux bruits, tremblements, etc).

6) Améliorations apportées

La caméra D435 est très sensible, notamment aux bruits et aux vibrations. Cela signifie que lorsque la caméra et l'objet ne bougent pas, les distances peuvent quand même varier. J'ai donc ajouté le lissage par EMA (Exponential Moving Average) : c'est une méthode qui permet de lisser les valeurs lorsqu'elles fluctuent trop. On se sert de l'ancienne valeur et de la nouvelle pour avoir la valeur voulue. Après plusieurs tests, j'ai choisi une valeur de 0.2 qui est un bon compromis entre réactivité et stabilité.

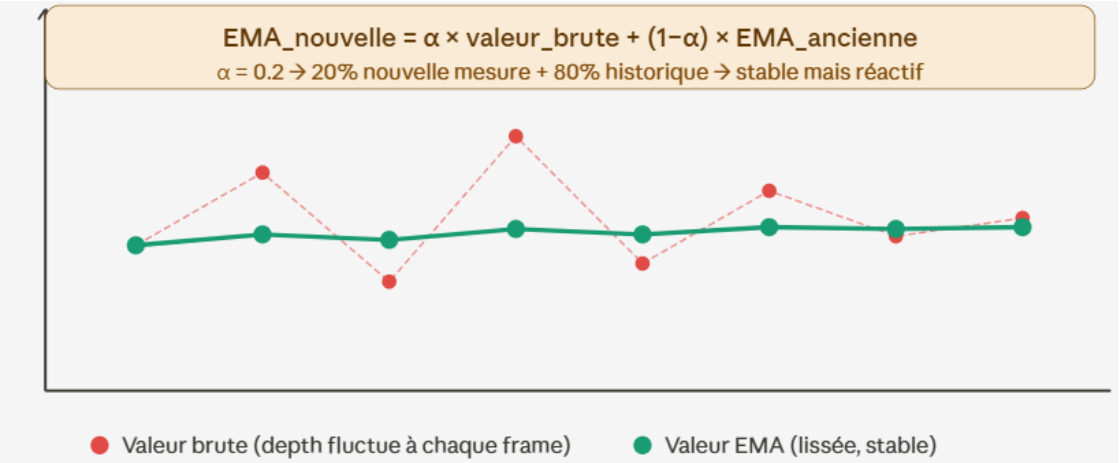

Figure 3 : Lissage par EMA

En outre, la taille de l'objet était calculée grâce aux délimitations de la bounding box. Le code faisait la différence de pixels entre x_{min}, x_{max} (abscisses) et y_{min},y_{max} (ordonnées). Or, si l'objet était incliné et qu'il n'était pas de la forme carrée de la bounding box, la taille trouvée n'était pas la taille réelle. J'ai donc ajouté une fonction qui permet d'ajuster et d'incliner la bounding box selon l'orientation de l'objet.

De plus, j'ai ajouté le centroïde de l'objet et ses coordonnées par rapport au centre de la caméra. Cette caractéristique est utile pour se repérer et sera d'une grande aide pour le futur bras du robot.

Finalement, la caméra D435 a des paramètres intrinsèques, c'est-à-dire qu'une calibration est faite en usine. Cependant, chaque lentille peut être légèrement différente. Ainsi, j'ai décidé de parfaitement calibrer ma caméra grâce à un damier : je l'ai imprimé, j'ai mesuré la taille d'une case et j'ai reporté les mesures dans un code. Celui-ci m'a créé un fichier .json qui contient des valeurs de calibration dont le code se sert. Créer ce fichier une seule fois suffit et le code de détection l'appellera à chaque lancement.

A ce stade, j'avais un code qui détectait les objets et qui était optimisé. J'avais donc réussi mon premier objectif et il me restait un peu de temps pour implémenter tout cela dans un environnement ROS2.

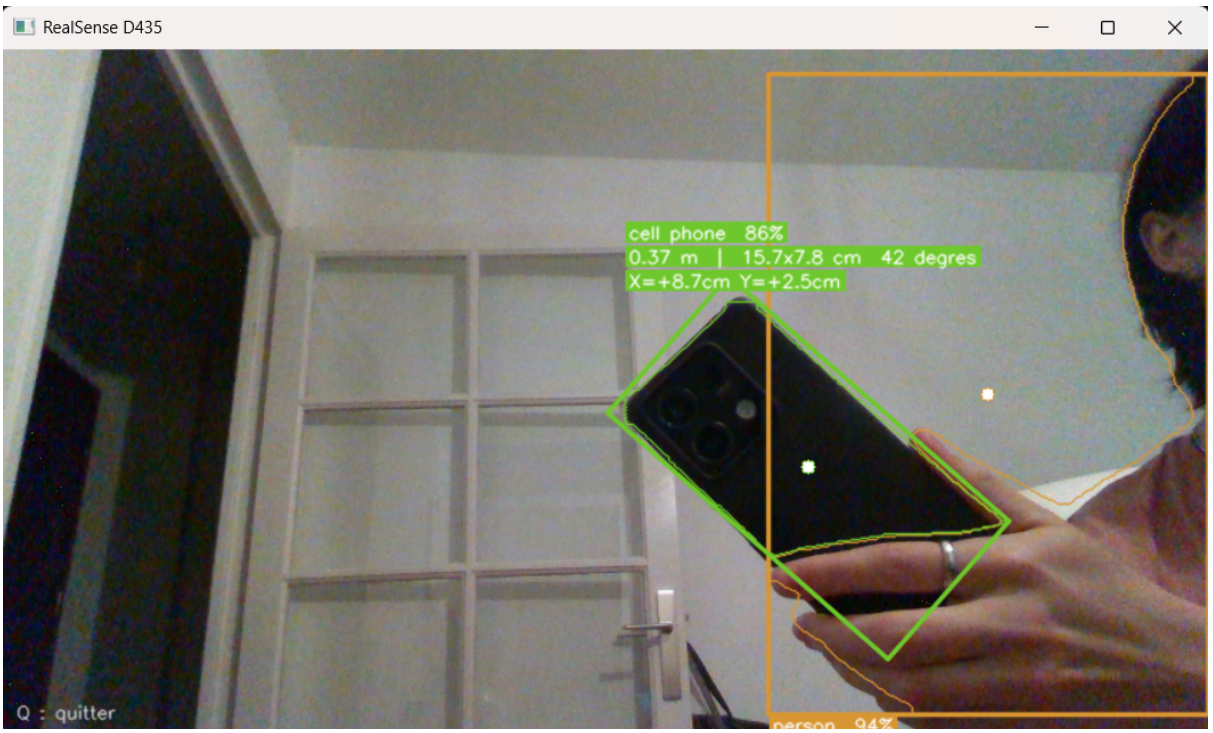

Figure 4 : Code optimisé de détection d'objets

7) Mise en place de l'environnement ROS2

Mon ordinateur étant sous Windows et n'ayant pas de Dual Boot Linux Ubuntu, j'ai utilisé WSL et Ubuntu 22.04 (seule version qui supporte ROS2 Humble). J'ai suivi un tutoriel d'installation de ROS2 sur mon ordinateur.

Après cela, j'avais donc un environnement de travail fonctionnel et j'ai pu commencer à travailler et à importer mes codes. J'ai cependant préféré créer un docker container pour isoler mon environnement de travail, dans lequel j'ai réinstallé toutes les librairies nécessaires.

Enfin, j'ai rencontré un problème de port USB avec la caméra : je ne pouvais pas la connecter à WSL puisqu'elle était connectée à Windows. J'ai effectué quelques recherches et suis tombé sur un GitHub avec un module USBIPD (cf. V/Bibliographie). Il est très simple d'utilisation et m'a permis de connecter mes ports USB à n'importe quel système de mon ordinateur avec une simple ligne de commande dans un terminal.

C'est ainsi que j'ai mis en place tout mon environnement de travail.

8) Implémentation dans ROS2

Pour l'implémentation dans ROS2, j'ai préféré garder les codes Python que j'avais. J'ai pu faire cela en créant un package de type *ament_python*.

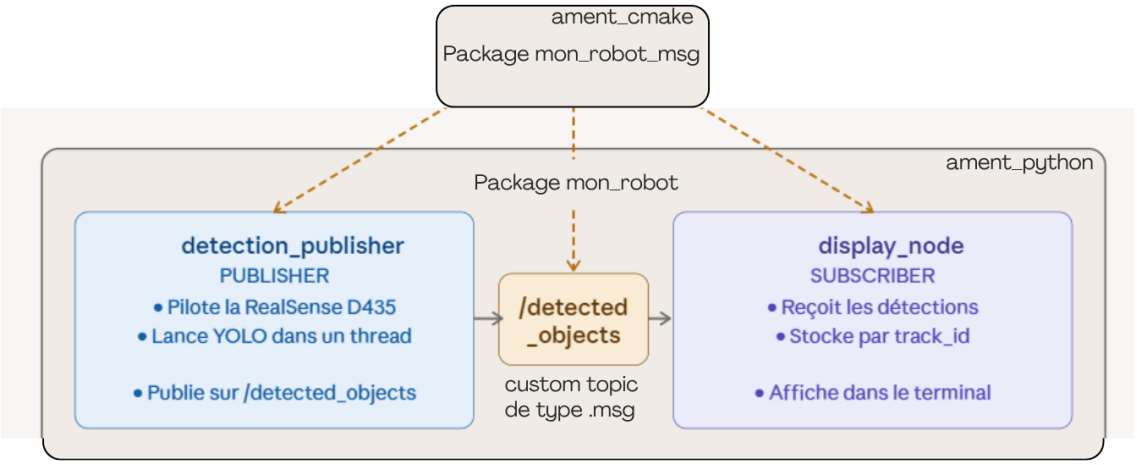

Figure 5 : Architecture de mon environnement ROS2

Je voulais un terminal qui affiche les objets détectés, il me fallait donc un nœud d'affichage. J'avais également besoin d'un nœud de détection dans lequel les codes de détection étaient implémentés. Puisque j'avais déjà tous mes codes Python, il m'a seulement fallu les adapter pour en faire deux nœuds. Le package *mon_robot* de type *ament_python* m'a permis de ne pas tout réécrire en C++. Ainsi, le noeud *detection_publisher* publie et le noeud *display_node* s'abonne.

Cependant, pour récupérer les valeurs voulues (la hauteur, la largeur, la distance, l'id, la classe de l'objet, etc.), il me fallait créer un custom topic. Mais pour cela, il me fallait un package de type *ament_cmake*. J'ai donc créé mon propre package *mon_robot_msg* qui contient le custom topic (de type *.msg*) *detected_objects.msg* avec toutes les informations voulues.

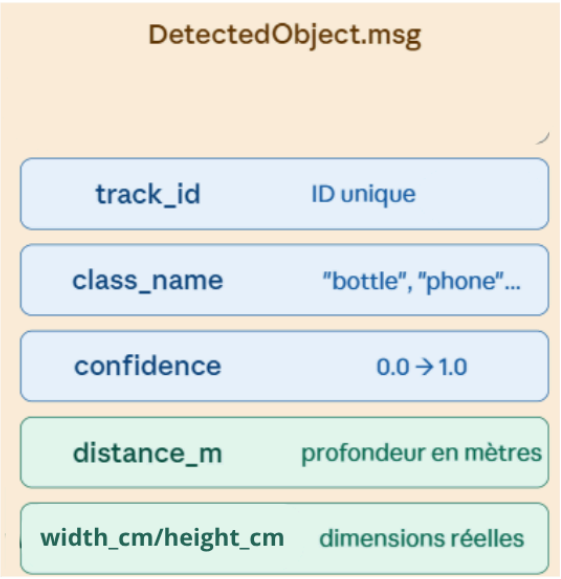

Figure 6 : Custom Topic

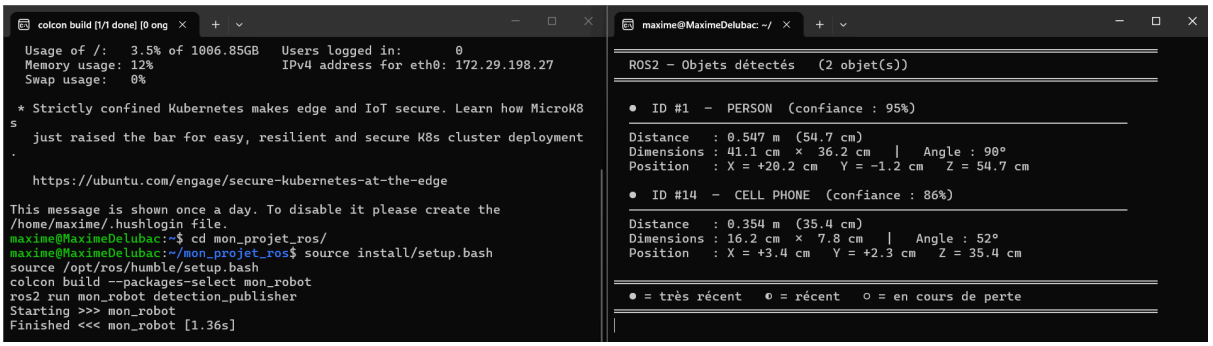

Figure 7 : Programme de détection d'objets ROS2

Pour lancer ce programme je dois :

- Connecter la caméra à WSL avec USBIPD,
- Ouvrir et lancer mon docker container,
- Ouvrir deux terminaux Ubuntu,
- Sourcer ROS2 dans chacun des terminal,
- Build le package *mon_robot*,
- Lancer un nœud dans chaque terminal.

9) Améliorations possibles

Il serait intéressant d'approfondir les aspects suivants :

- Je pourrais facilement implémenter un bras dans cette architecture. En effet, il lui suffirait de s'abonner au custom topic et de récupérer les informations nécessaires comme la distance et la position du centroïde. Cependant, le code du mouvement du bras et le fonctionnement de la pince pour ramasser l'objet n'étant pas trivial, il me faudrait plus de temps pour l'implémenter.
- Une solution beaucoup plus optimale pour de l'embarqué serait de prendre un modèle d'IA moins lourd ou un réseau de neurones (CNN) non entraîné, et l'entraîner sur des objets adaptés à ce projet. YOLOv8 détecte des centaines d'objets différents et la grande majorité (par exemple, une voiture ou une chaise) ne sont pas utiles dans notre cadre.
- Je pourrais convertir mes codes en C++ pour pouvoir les mettre dans un package ROS2 de type *ament_cmake*. Cela me permettrait d'avoir un seul package. De plus, le C++ compile plus rapidement que Python et est un langage universel dans le domaine de la robotique. Cependant, en C++, il est plus difficile d'utiliser et d'importer Yolo qu'avec Python.

IV/ Conclusion

Ce projet m’a permis de faire le lien entre mes cours théoriques en spécialité Robotique et l’application directe sur un robot.

J’ai fortement développé mes compétences en Machine Vision et ai pu prendre en main et utiliser des modèles de réseaux de neurones pré-entraînés comme YOLOv8.

Le passage à une caméra de profondeur m’a offert plus de possibilités que lors de mon premier projet. J’ai pu par exemple exploiter les données du flux profondeur, ce que je ne pouvais pas faire avec une caméra RGB classique.

La création d’un environnement ROS2 m’a permis de plus m’imprégner dans la robotique puisqu’il est fortement utilisé dans ce domaine. De plus, grâce aux cours de ROS2 auxquels j’ai participé en cours d’année, et lors desquels j’ai utilisé le bras PX-100, je pourrais implémenter le fonctionnement de ce dernier dans mon deuxième projet.

V/ Bibliographie

Lien GitHub du premier projet :
https://github.com/MaximeDelubac/Delubac_ROB4_Bras_Robot

Lien GitHub du second projet :
https://github.com/MaximeDelubac/Delubac_ROB4_Cam-ra_d435_IntelRealSense

Lien GitHub de USBIPD :
https://github.com/dorssel/usbipd-win